

Sistema de Login & Sessão

Consultório (app CustomTkinter) — arquitetura, fluxo, segurança e o bug de janela que foi corrigido. Documento gerado em 28/09/2026.

Sumário

1. Visão geral — do `python app.py` até a agenda
2. Arquitetura de janela única e o bug corrigido
3. Componentes do sistema de login
4. Fluxo passo a passo (inicialização, login, logout)
5. Manter conectado (Credential Manager)
6. Multi-usuário: isolamento por `usuario_id`
7. Segurança
8. Checklist de teste manual

1. Visão geral

O login é a primeira tela do aplicativo. Ao executar `python app.py`, a função `iniciar()` (`app.py:207`) cria **uma única janela** (a classe `App`, que herda de `ctk.CTk`) e então decide o que mostrar:

- Existe uma **sessão válida** salva no Cofre do Windows? Chama `app.entrar(usuario)` e o usuário cai direto na agenda.
- Não existe (ou o usuário foi apagado)? Mostra a tela de login dentro da mesma janela.

A janela raiz **nunca é destruída** enquanto o programa roda. Login, agenda, pacientes, orçamento... tudo são telas desenhadas dentro do mesmo `main_frame`. Só ao fechar o programa é que a janela morre. Essa decisão de projeto é a resposta a um erro que já aconteceu — ver a seção 2.

2. Arquitetura de janela única e o bug corrigido

2.1 A regra de ouro do Tk

Tk (a biblioteca gráfica por baixo do CustomTkinter) **não tolera** destruir e recriar a janela raiz dentro do mesmo processo. Quando a última `ctk.CTk()` é destruída, o interpretador Tcl inteiro morre; timers pendentes (`after()`) deixados em widgets mortos explodem depois. No Consultório: **uma única raiz; as telas trocam o conteúdo**.

2.2 Como era (e porque quebrava)

Antes da correção, a tela de login era uma janela separada e o fluxo destruía/recriava janelas no login e no logout:

ANTIGO

```
# views/login.py (ANTIGO)
janela = ctk.CTk() # criava uma JANELA RAIZ própria
janela.title("Consultório – Login")
...
return janela # app.py chamava janela.mainloop()

# app.py (ANTIGO)
def abrir_app(usuario):
    definir_usuario(usuario)
    if janela_login is not None:
        janela_login.destroy() # destruía a raiz do login
    app = App() # criava OUTRA raiz no mesmo processo
    app.mainloop()

def sair(self): # botão "Sair"
    limpar_sessao()
    self.voltou_para_login = True
    self.destroy() # destruía a raiz do App; iniciar() se
    ... # chamava de novo (recursão)
```

2.3 Os erros que apareciam no console

Erro	Causa
invalid command name "...check_dpi_scaling"	O monitor de DPI do CustomTkinter (scaling_tracker.py) agenda um loop com after() na janela raiz. Raiz destruída -> o comando do callback morre junto; quando o timer dispara, o Tcl não acha mais nada.
invalid command name "...update"	Mesma origem: loop interno do CustomTkinter agendado na janela já destruída.
invalid command name "..._click_animation"	O botão agenda after(100, ...) ao ser clicado; se o botão for destruído antes (login -> entrar), o callback fica órfão.
_tkinter.TclError: can't invoke "wm" command: application has been destroyed	Destruir a última janela raiz derruba o app Tcl inteiro. O novo App() nascia já morto e quebrava ao montar a janela.

Ou seja: **o erro não era de lógica de negócio** — nenhuma senha, banco ou consulta estava errada. O problema era estrutural: cada login e cada logout destruía uma janela raiz, e o Tk deixa sujeira pendente quando isso acontece.

2.4 A correção

NOVO

```
# views/login.py (NOVO) – virou view normal, como as outras telas
def mostrar(parent, on_success=None):
    # desenha DENTRO do main_frame do App; nenhuma janela nova
    container = ctk.CTkFrame(parent, fg_color="transparent")
    container.pack(fill="both", expand=True)
    frame_login = ctk.CTkFrame(container, width=420, height=520, ...)
    ...

# app.py (NOVO) – a raiz sobrevive ao logout
def sair(self):
    limpar_sessao() # apaga a credencial
    definir_usuario(None) # zera o contexto
    self.logado = False
    self.sidebar.pack_forget() # esconde o menu (sem destroy)
    self.mostrar_login_tela() # volta a mostrar o login
```

Mudanças principais:

- `views/login.py` deixou de criar `ctk.CTk()` — agora recebe `parent` e desenha o cartão de 420x520 centralizado.
- `app.py` ganhou o ciclo de vida: `entrar()`, `mostrar_login_tela()`, `sair()` sem `destroy()` da raiz; saiu a recursão do antigo `iniciar()`.
- A sidebar só é empacotada após o login (antes: era criada já visível); o relógio de consultas só consulta o banco com `self.logado == True`.

2.5 Validação

- `ruff check .` limpo e **98 testes** passando.
- Script automatizado de ciclo completo (login -> clicar Entrar -> Sair -> login -> fechar, 2x) com `stderr` **vazio** e `exit code 0`.
- Micro-testes isolados confirmaram: um `after()` órfão de widget destruído vira um erro de fundo **silencioso** do Tk (sem `bgerror`, o Tcl engole) — nada é impresso e nada quebra.

3. Componentes do sistema de login

Arquivo	Função	O que faz
<code>app.py</code>	<code>iniciar()</code>	Ponto de entrada. Cria o App (raiz única), lê a sessão salva no keyring, decide entre <code>entrar()</code> e <code>mostrar_login_tela()</code> , depois chama o <code>mainloop()</code> .
<code>app.py</code>	<code>App.entrar(usuario)</code>	Autentica: grava o usuário no contexto, mostra a sidebar, abre a agenda e liga o relógio de consultas (só na primeira vez).
<code>app.py</code>	<code>App.mostrar_login_tela()</code>	Limpa o <code>main_frame</code> e desenha a tela de login dentro dele.
<code>app.py</code>	<code>App.sair()</code>	Logout: apaga a credencial, zera o contexto, esconde a sidebar e volta ao login. Não destrói nenhuma janela.
<code>app.py</code>	<code>App.verificar_horarios_consultas()</code>	Relógio de 30s que procura consulta no bloco de horário atual e abre o pop-up de comparecimento. Consulta o banco somente se <code>self.logado</code> .
<code>views/login.py</code>	<code>mostrar(parent, on_success)</code>	Desenha o cartão de login (título, e-mail, senha, 'Manter conectado', botões Entrar e Criar Usuário) dentro do container recebido.
<code>views/login.py</code>	<code>tentar_login()</code>	Valida os campos, busca o usuário, compara a senha, salva ou limpa a sessão e chama <code>on_success(usuario)</code> (= <code>App.entrar</code>).
<code>database/contexto.py</code>	<code>definir_usuario / usuario_atual / usuario_id_obrigatorio</code>	Contexto global do usuário logado. Toda função de banco usa <code>usuario_id_obrigatorio()</code> ; sem login, levanta <code>RuntimeError</code> .
<code>database/sessao.py</code>	<code>salvar / carregar / limpar_sessao</code>	Persiste só o número do ID no Credential Manager do Windows (serviço consultório).
<code>database/usuarios.py</code>	<code>get_user_by_email / get_user_by_id / validar_senha</code>	Consulta a tabela <code>Usuarios</code> e compara o hash da senha.
<code>views/usuarios.py</code>	<code>abrir_criar_usuario(parent)</code>	Pop-up (<code>CTkToplevel</code>) de criação de conta, acessível pela tela de login.

4. Fluxo passo a passo

4.1 Inicialização

```
python app.py
|
v
iniciar() app.py:207
|-- carregar_sessao() (keyring: tem ID salvo?)
| |
| | sim não / expirado
| | v v
| | get_user_by_id(uid) mostrar_login_tela()
| | | |
| | existe apagado [TELA DE LOGIN]
| | | |
| | limpar_sessao() | usuário digita e-mail + senha
| | | e clica "Entrar" (ou Enter)
| | '----> [TELA DE LOGIN] |
| | |
| | '-----'-----> app.entrar(usuario) <-'
|
v
app.mainloop() <== raiz ÚNICA: destruída
somente ao fechar o programa
```

4.2 Login (botão Entrar)

```
tentar_login() views/login.py:72
|-- campos vazios? -> "Preencha e-mail e senha."
|-- erro no banco? -> "Erro ao acessar o banco."
|-- e-mail não cadastrado? -> "Usuário não encontrado."
|-- senha não confere? -> "Senha incorreta."
'-- tudo certo
|-- "Manter conectado" MARCADO? -> salvar_sessao(id) [keyring]
| (desmarcado) -> limpar_sessao()
|-- on_success(usuario) = App.entrar(usuario)
|-- definir_usuario(usuario) contexto global
|-- sidebar.pack(...) aparece o menu
|-- limpar_frame() + agenda troca de tela
'-- relógio de consultas (1ª vez)
```

4.3 Logout (botão Sair)

```
app.sair() app.py:198
|-- limpar_sessao() apaga a credencial do Cofre do Windows
|-- definir_usuario(None) zera o contexto de usuário
|-- self.logado = False relógio para de consultar o banco
|-- sidebar.pack_forget() esconde o menu (widget permanece vivo)
'-- mostrar_login_tela() volta à tela de login
(a janela raiz continua de pé – zero destroy)
```

5. Manter conectado

O checkbox "**Manter conectado**" vem **marcado por padrão**. Quando o usuário loga assim, apenas o **número do ID** da conta é gravado no **Credential Manager do Windows** (via biblioteca keyring, serviço consultório, conta `usuario_id`).

- **Nunca é salva senha** — só o ID; na próxima abertura o app busca o registro no banco com `get_user_by_id()`.
- Se o usuário desmarcar a caixa e logar, a credencial é **apagada** (`limpar_sessao()`).
- Se o Cofre estiver indisponível ou o registro não existir, `carregar_sessao()` retorna `None` sem quebrar: o app simplesmente mostra o login.
- Se o ID salvo apontar para um usuário que foi excluído, `iniciar()` limpa a sessão e mostra o login.
- O botão **Sair** sempre apaga a credencial — próxima abertura pede login.

6. Multi-usuário: isolamento por `usuario_id`

Cada conta enxerga apenas os próprios dados. As tabelas `Paciente`, `Consulta`, `Orcamento` e `Tratamento` têm a coluna `usuario_id` (FK para `Usuarios.id`).

- As funções de banco mantiveram as mesmas assinaturas: cada uma chama `usuario_id_obrigatorio()` e monta o filtro com `_filtro_usuario()`. As telas não precisaram mudar.
- Sem usuário logado, a função lança `RuntimeError("Nenhum usuário logado no contexto...")` — falha alto e explícito, nunca vaza dado.
- **É um usuário por processo:** o contexto é uma variável global em memória. "Multi-usuário" aqui significa **isolamento de dados** (trocar de conta limpa e recarrega tudo), não uso simultâneo.
- Trocar de conta = Sair -> logar de novo: o `limpar_frame()` garante que nenhuma tela anterior sobreviva com dados do usuário antigo.

7. Segurança

- **Senha:** guardada apenas como hash (`senha_hash`); `validar_senha()` compara e nunca exhibe o valor.
- **Sessão:** só um número no Cofre do Windows — nenhum arquivo de sessão no diretório do projeto, nada legível em texto.
- **Contexto:** o usuário logado vive só na memória do processo; `sair()` zera.
- **Nota:** a senha de administração do Supabase aparece em texto nos históricos de migração (`melhorias/PASSOS_LOGIN_CLOUD.txt` e no histórico do git) — recomenda-se trocar essa senha no painel do Supabase.

8. Checklist de teste manual

1. Abrir `python app.py` sem sessão: UMA só janela, login centralizado, console sem erros.
2. Campos vazios + Entrar: Preencha e-mail e senha.
3. E-mail inexistente: Usuário não encontrado.; senha errada: Senha incorreta.
4. Login válido com 'Manter conectado' marcado: sidebar aparece, agenda abre, console limpo.
5. Fechar e reabrir o app: entra direto na agenda (sessão restaurada).
6. Sair: volta ao login; a credencial foi apagada (próxima abertura pede login).
7. Logar com a caixa desmarcada: fechar e reabrir pede login de novo.
8. Trocar de conta (Sair -> outro usuário): agenda/pacientes mostram só dados do novo usuário.
9. Botão 'Criar Usuário': pop-up abre sem erro Tcl; concluir e logar com a conta nova.
10. Ciclo completo no console: NENHUM `invalid command name`, `wm` ou `TclError`.

Próximos passos: login com Google (`google-auth`) — decisão registrada: qualquer conta Gmail cria automaticamente uma conta no sistema.